

LV03 Direct Addressing Table, Hash Table, 패턴찾기

■ 소유자	암암코딩
■ 태그	
■ 생성 일시	@2023년 12월 11일 오후 7:40

패턴찾기

vect

B	T	A	B	C	Q	A	B	C
---	---	---	---	---	---	---	---	---

pattern

A	B	C
---	---	---

Flag 써서 찾거나, 함수를 만들어서 찾는다

```
#include <iostream>
using namespace std;

char vect[12] = "BTABCCQABC";
char pattern[4] = "ABC";

bool isPattern(int startIdx)
{
    for (size_t i = 0; i < 3; i++)
    {
        if (pattern[i] != vect[startIdx + i])
        {
            return false;
        }
    }
    return true;
}
```

```

        return false;
    }
}

return true;
}

int main()
{
    for (size_t i = 0; i < 7; i++)
    {
        if (isPattern(i) == true)
        {
            cout << "Pattern found at index: " << i << endl;

            break;
        }
    }

    return 0;
}

```

2차원배열에서도 패턴을 찾을수 있다.

vect	0	3	5	1
	1	2	4	5
	2	3	6	1
	3	3	1	1
	4	4	5	6
	5	2	4	5
pattern		2	4	5

```
#include <iostream>
using namespace std;
```

```
int vect[6][3] =
{
    3, 5, 1,
    2, 4, 5,
    3, 6, 1,
    3, 1, 1,
    4, 5, 6,
    2, 4, 5
};

int pattern[3] = { 2, 4, 5 };

int isPattern(int line)
{
    for (int x = 0; x < 3; x++)
    {
        if (pattern[x] != vect[line][x]) return 0;
    }
    return 1;
}
```

```
int main()
{
    int result;

    for (int y = 0; y <= 5; y++)
    {
        result = isPattern(y);
        if (result == 1) break;
    }

    if (result == 1) cout << "발견";
    else cout << "미발견";

    return 0;
}
```

또다른 패턴찾는 예시

vect	1	2	3	4	1
	3	1	0	0	1
	2	3	4	1	2
pattern	3	4	1		

```

#include <iostream>
using namespace std;

int vect[3][5] =
{
    1,2,3,4,1,
    3,1,0,0,1,
    2,3,4,1,2,
};
int pattern[3] = {3,4,1};

bool isPattern(int dy, int dx)
{
    for (size_t i = 0; i < 3; i++)
    {
        if (pattern[i] != vect[dy][dx + i])
        {
            return false;
        }
    }

    return true;
}

int main()
{

```

```

bool ret = false;
for (int y = 0; y < 3; y++)
{
    for (size_t x = 0; x < 3; x++)
    {
        ret = isPattern(y, x);
        if (ret)
            break;
    }

    if (ret == true)
    {
        // 존재
    }
    else
    {
        // 존재하지 않음
    }
}

return 0;
}

```

패턴이 게임에서 활용되는 예시의 대표적으로 스타듀밸리가 있다.

				
물뿌리개	구리 물뿌리개	강철 물뿌리개	황금 물뿌리개	이리듐 물뿌리개
-	2,000G 구리 주괴 5개	5,000G 철 주괴 5개	10,000G 금 주괴 5개	25,000G 이리듐 주괴 5개
				

해시 테이블

해시 테이블이란 해시함수를 사용하여 변환한 값을 색인(index)으로 삼아 키(key)와 데이터(value)를 저장하는 자료구조를 말한다. 기본연산으로는 탐색(Search), 삽입(Insert), 삭제(Delete)가 있다.

Direct Address Table

먼저 가장 간단한 형태의 해시테이블로 이름 뜻대로 키 값을 주소로 사용하는 테이블을 말한다. 이는 키 값이 100이라고 했을 때 배열의 인덱스 100에 원하는 데이터를 저장하는 것이다.

```
int bucket[256] = {};

char str[7] = "ADBFAD";
for (int i = 0; i < 6; i++)
{
    bucket[str[i]]++; //str[i](아스키코드)값 자체를 index로 활용
}
```

Hash Table - DAT(Direct Addressing Table)

```

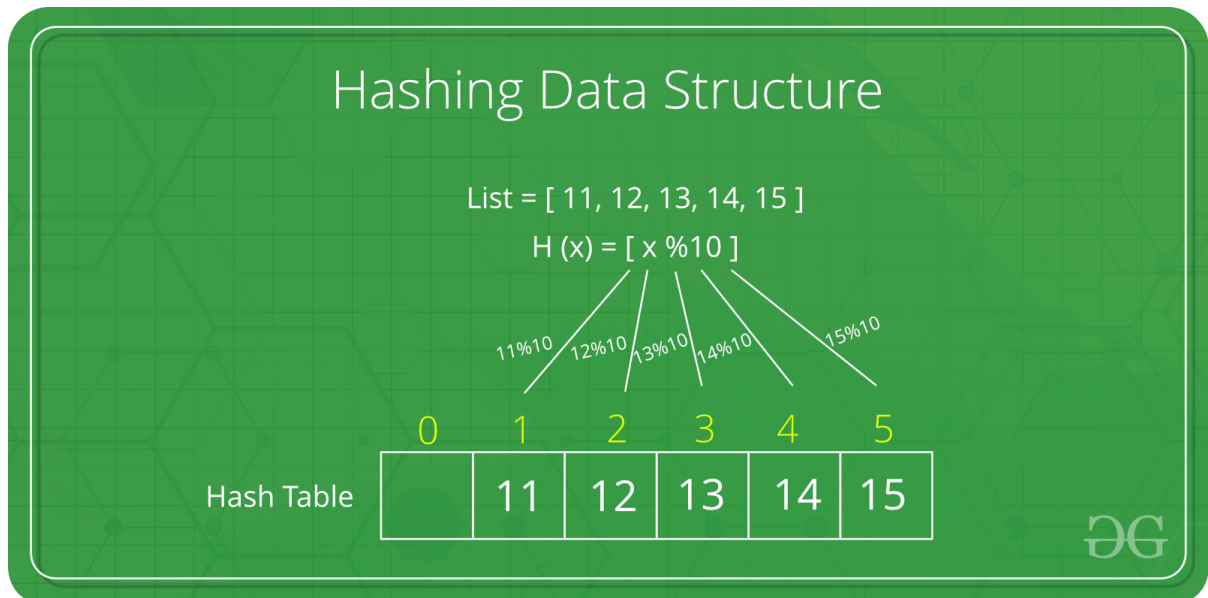
#include <iostream>

int main()
{
    // DAT(Direct Addressing Table) - Hash Table
    // 값 자체를 index로 활용해서 사용하겠다.
    // 넉넉한 크기의 배열을 선언
    int bucket[256] = {};
    char target = 'A';
    // 'A'의 아스키코드는 65
    // bucket 배열의 65번째에 1을 대입
    bucket[target] = 1;

    return 0;
}

```

Hash Table



해시함수를 사용하여 특정 해시값을 알아내고 그 해시값을 인덱스로 변환하여 키 값과 데이터를 저장하는 자료구조이다. 이는 보통 알고 있는 해시 테이블을 얘기하며 개념자체가 어려

운 것은 아니지만 문제가 되는 것은 **충돌(Collision)**이다. 충돌에 대해서 이해하기 위해선 먼저 **적재율(Load Factor)**에 대해서 이해해야 한다.

적재율이란 해시 테이블의 크기 대비, 키의 개수를 말한다. 즉, 키의 개수를 K , 해시 테이블의 크기를 N 이라고 했을 때 적재율은 K/N 이다. Direct Address Table은 키 값을 인덱스로 사용하는 구조이기 때문에 적재율이 1 이하이며 적재율이 1 초과인 해시 테이블의 경우는 반드시 충돌이 발생하게 된다.

만약, 충돌이 발생하지 않다고 할 경우 해시 테이블의 탐색, 삽입, 삭제 연산은 모두 $O(1)$ 에 수행되지만 충돌이 발생할 경우 탐색과 삭제 연산이 최악에 $O(K)$ 만큼 걸리게 된다. 이는 같은 인덱스에 모든 키 값과 데이터가 저장된 경우로 충돌이 전부 발생했음을 말한다. 따라서, 충돌을 최대한으로 줄여서 연산속도를 빠르게 하는 것이 해시 테이블의 핵심인데 이에 중요하게 작용하는 것이 바로 해시함수를 구현하는 **해시 알고리즘**이다. 해시 알고리즘이 견고하지 못하게 되면 해시함수로 도출된 값들이 같은 경우가 빈번하게 발생하게 되므로 잦은 충돌로 이어지게 되는 것이다.

활용의 예

배열에 존재하는 알파벳 갯수 찾기

```
#include <iostream>

int main()
{
    // 넉넉한 크기의 배열을 선언
    int bucket[256] = {};
    char str[7] = "ADBFAD";

    for (int i = 0; i < 6; i++)
    {
        // 아스키코드 자체를 인덱스로 사용
        int idx = str[i];
        bucket[idx]++;
    }

    // for문을 사용해서 하나하나 count하여 찾을 수도 있다.
```



```

int count = 0;
for (int i = 'A'; i <= 'Z'; i++)
{
    std::cout << (char)i << " : " << bucket[i] << std::endl;
}

// 2가지 방법의 장단점이 존재한다.
// HashTable을 사용하면 너무 많은 메모리 공간을 차지한다. 대신 코드는 쉬워진
다.
// for문을 사용하면 코드가 복잡하다. 대신 정해진 공간만 사용한다.

return 0;
}

```

배열에 존재하는 알파벳의 종류 찾기

```

#include <iostream>

int main()
{
    // 넉넉한 크기의 배열을 선언
    int bucket[256] = {};
    char str[7] = "ADBFAD";

    for (size_t i = 0; i < 6; i++)
    {
        // 아스키코드 자체를 인덱스로 사용
        // 알파벳 갯수를 확인
        char idx = str[i];
        bucket[idx]++;
    }

    for (size_t x = 0; x < 256; x++)
    {
        // 알파벳이 존재한다면
        if (bucket[x] != 0) {
            // 사용된 알파벳의 종류가 출력된다.
            std::cout << (char)x;
        }
    }
}

```

```

    }
}

return 0;
}

```

배열에 존재하는 알파벳의 종류와 각 갯수 출력

```

#include <iostream>

int main() {
    // 넉넉한 크기의 배열을 선언
    int bucket[256] = {};
    char str[7] = "ADBFAD";

    for (size_t i = 0; i < 6; i++)
    {
        // 아스키코드 자체를 인덱스로 사용
        // 알파벳 갯수를 확인
        bucket[str[i]]++;
    }

    for (size_t x = 0; x < 256; x++)
    {
        // 알파벳이 존재한다면
        if (bucket[x] != 0) {
            // 사용된 알파벳의 종류가 출력된다.
            // 사용된 알파벳의 종류 별 갯수도 함께 출력한다.
            std::cout << (char)x << " : " << bucket[x] << "개\n";
        }
    }

    return 0;
}

```

배열의 글자를 순서대로 정렬하여 출력

버블정렬, 선택정렬보다 훨씬 간단하다.

```
#include <iostream>

int main()
{
    // 넉넉한 크기의 배열을 선언
    int bucket[256] = {};
    char str[7] = "ADBFAD";

    for (size_t i = 0; i < 6; i++)
    {
        // 아스키코드 자체를 인덱스로 사용
        // 알파벳 갯수를 확인
        bucket[str[i]]++;
    }

    for (size_t x = 0; x < 256; x++)
    {
        // 알파벳이 존재한다면
        if (bucket[x] != 0)
        {
            // 사용된 알파벳을 정렬하여 출력
            for (size_t i = 0; i < bucket[x]; i++)
            {
                std::cout << (char)x;
            }
        }
    }

    return 0;
}
```

패턴 찾기

```
#include <iostream>
```

```
char vect[10] = "BTABCQABC";
char pattern[4] = "ABC";
// 문자열 배열은 끝났을 때 어디서 끝나는지 알려줘야 하기 때문에
// 마지막 문자인 '\0' null 문자의 자리까지 포함해서 문자열 배열의 사이즈를 선언한
다.
// char pattern[4] = "ABC\n" 라는 값을 가진 것이다.
```

```
int isPattern(int idx) {
    for (size_t i = 0; i < 3; i++)
    {
        if (pattern[i] != vect[idx + i]) {
            return 0;
        }
    }
    return 1;
}
```

```
int main() {
    // 패턴찾기 (함수 이용)
    int result = 0;
    for (size_t i = 0; i < 7; i++)
    {
        result = isPattern(i);
        if (result == 1) {
            break;
        }
    }

    if (result == 0) {
        std::cout << "발견";
    }
    else {
        std::cout << "미발견";
    }

    return 0;
}
```

안녕하세요,

야무지고 맛있게 코딩하는 **얌얌코딩**입니다!

단순히 영상을 보는 것만으로는 부족한 분들,

"직접 손으로 풀어보며 성장하고 싶다"는 분들을 위해

얌얌코딩 유튜브 멤버십 전용 혜택을 마련했습니다.



멤버십 전용 혜택

단계별 연습문제 & 복습문제 PDF 제공

→ 강의에서 배운 내용을 직접 연습하며 복습할 수 있어요!

디스코드 전용 채널 입장 가능

→ 질문, 피드백, 코드 리뷰, 실습 자료 공유까지

👉 [디스코드 서버 바로가기](#)

선공개 자료 및 전용 코드 예제 제공

커리어, 포트폴리오 관련 Q&A/코멘트도 비정기적으로 제공

⚠️ 얌얌코딩 숙제, 전용 자료실은 "학생얌얌이" 등급 이상부터 제공됩니다.

일반 멤버십(입문얌얌)은 접근이 제한되며,

연습문제 다운로드 및 디스코드 채널 열람은 "학생얌얌" 등급 이상이어야 가능합니다.



가입 방법

1. 아래 링크 클릭하여 멤버십 가입

👉 [멤버십 가입하기](#)

2. 디스코드 서버 입장

👉 [디스코드 초대 링크](#)

3. 유튜브 계정과 디스코드 계정 연동

- 자동으로 멤버 등급 확인 후, 전용 채널 열립니다
- (문제 시 디스코드 내 안내 채널에서 도움 요청 가능)

▼ oopy:hide

연습문제



문제 1번

cardList 배열에 여러종류의 카드가 있습니다.

아래와 같이 카드를 입력받아주세요 (한 문장을 입력받아주세요, 최대 15글자)

DAT(Direct Address Table)을 이용하여 총 몇 종류의 카드가 있는지 count 해서 출력 해주세요.

CardList

A	B	C	D	A	C	A	B	C	D	E
---	---	---	---	---	---	---	---	---	---	---

출력: 5개

입력 예제

ABCDACABCDE

출력 결과

5개



문제 2번

1~65535 번의 ID를 가진 사람들이 있습니다.

출입기록이 있을 때 **가장 성실하게 출근한 사람이 누구지** 알려주는 프로그램을 작성해주세요.

(Direct Address Table을 사용해서 풀어주세요, 입력값은 없습니다)

출입기록

65000	35	42	70
70	35	65000	1300
65000	30000	38	42

출력: 65000

출력 결과

65000



문제 3번

9개 숫자를 입력받고, 3x3 배열에 채워주세요.

1~9까지 숫자 중 어떤 숫자들이 없는지를 찾아서 출력 해주세요.

ex)

입력:

3 5 5

1 4 9

2 2 1

3	5	5
1	4	9
2	2	1

출력: 6 7 8

(없는 숫자는 6, 7, 8 입니다)

입력 예제

3 5 5

1 4 9

2 2 1

출력 결과

6 7 8



문제 4번

8글자를 입력 받아주세요.

가장 많이 나온 알파벳을 찾는 함수를 만들어 주세요.

해당 반환값을 받아 출력해주세요.

(1등은 한명만 존재합니다.)

ex)

B	Q	T	A	B	A	B	A
---	---	---	---	---	---	---	---

입력 : BQTABABA

출력: A

입력 예제

BQTABAAA

출력 결과

A



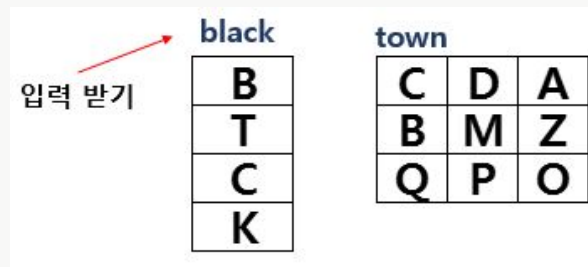
문제 5번

3×3 town배열을 하드코딩 해주세요.

블랙리스트 네명을 입력 받고,

마을에 블랙리스트에 해당하는 사람이 몇 명 있는지 출력하세요.

ex)



입력 : BTCK

출력: 2명

입력 예제

BTCK

출력 결과

2명



문제 6번

칸에 알파벳들이 채워져 있습니다.

이 알파벳들을 정렬해서 한줄로 출력해주세요.

(DirectAddressTable를 사용해서 풀어주세요)

A	B	C
A	G	H
H	I	J
K	A	B
A	B	C

출력: AAAABBBCCGHHIJK

출력 결과

AAAABBBCCGHHIJK



문제 7번

train

3	7	6	4	2	9	1	7
---	---	---	---	---	---	---	---

← 하드코딩

team

7	6	4
---	---	---

← 입력

기차(train)에 탄 사람들 중에 우리팀이 있는지 찾으려 합니다.

우리팀(team)을 입력 받고

우리팀이 기차에서 **몇번부터~몇번째 칸에** 탑승하고 있는지 출력 하세요.

입력 예제

7 6 4

출력 결과

1번~3번 칸



문제 8번

아래는 아파트를 배열으로 나타낸 사진입니다.

맨 아래는 1층 (15 2 6)

맨 꼭대기층은 5층 (15 18 17) 입니다.

이 배열을 하드코딩 해 주세요.

15	18	17
4	6	9
10	1	3
7	8	9
15	2	6

숫자 3개를 family 배열에 입력 받으세요.

이 family가 몇층에 사는지 isPattern 함수를 이용해서 문제를 풀어 주세요.

입력 예제

7 8 9

출력 결과

2층



문제 9번

1	3	3	5	1
3	6	2	4	2
1	9	2	6	5

위 배열을 **하드코딩** 하세요.

그리고 숫자 1개를 변수 n에다가 입력 받으세요.

위 배열에는 똑같은 숫자가 여러개가 존재합니다.

예로들어 숫자 1은 3개 존재하고,

숫자6은 2개 존재합니다.

n개 존재하는 숫자를 출력하는 프로그램을 작성 해 주세요

예로들어 숫자 2를 입력받으면,

배열에서 2개 존재하는 숫자를 찾아야 합니다.

따라서 5와 6 을 출력 하시면 됩니다.

입력 예제

3

출력 결과

1 2 3



문제 10

Person 클래스를 설계하라. Person 클래스는 이름, 주소, 전화 번호를 멤버 변수로 가진다.

하나의 상의 생성자를 정의하고 각 멤버 변수에 대하여 Get와 Set 함수를 작성하라.

```
이름을 입력하세요 :김철수
주소를 입력하세요 :서울시
연락처를 입력하세요 :010-1101-2202
ID를 입력하세요 :110
마일리지를 입력하세요 :1500
-----고객 정보-----
이름: 김철수
주소: 서울시
연락처: 010-1101-2202
고객ID: 110
마일리지: 1500
```

그리고 이미지와 같이 객체를 하나 생성한후 이름, 주소, 연락처, ID, 마일리지를 입력받고

출력하는 프로그램을 만들어라.

복습문제



문제 1번

G	K	G

문자 3개를 배열의 빈 칸에다가 입력 받고, 빈칸에 채워주세요.

총 6개의 문자에서 같은 문자가 3개이상(≥ 3) 존재 하는지 검사하는 소스코드를 작성해주세요.

존재하면 "있음" 출력

존재하지 않으면 "없음" 출력

예로들어 A B C를 입력받으면

배열에는 G K G A B C 이렇게 6개 문자가 있습니다.

여기서 같은문자 3개이상 존재하는 문자는 없으니

"없음" 이 답입니다.

입력 예제

A B C

출력 결과

없음



문제 2번

1	3	3	5	1	4
---	---	---	---	---	---

6개 숫자를 입력 받으세요.

같은 숫자가 존재하는지 확인하고 출력 해주세요.

존재O ⇒ "도플갱어 발견" 출력

존재X ⇒ "미발견" 출력

- 충분한 설계를 하신 후 풀어주시면 됩니다

입력 예제

1 3 3 5 1 4

출력 결과

도플갱어 발견



문제 3번

한문장을 입력 받으세요.(최대 10글자)

그리고 가장 많은 알파벳이 어떤 알파벳인지 출력 해주세요.

입력 예제

AKFBBQAAK

출력 결과

A



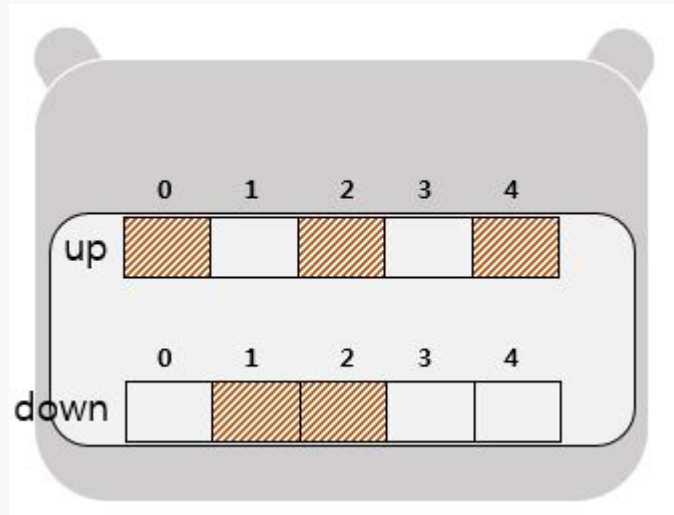
문제 4번

하마는 여러충치를 가지고 있는데, 입을 닫으면 이빨끼리 부딪칩니다.

이때 충치끼리 닿으면 하마가 아파합니다.

up 2번치아와 down 2번치아는 충치라서 닿으면 하마가 고통을 느낍니다.

윗니와 아랫니 상태를 입력받고, 고통을 느끼는 치아의 갯수를 출력 하세요.



입력 예제

1 0 1 0 1

0 1 1 0 0

출력 결과

1개



문제 5번

아래 문장을 하드코딩 해 주세요

ATKPTCABC

그리고 두 문자를 입력받아주세요

첫번째 문자는 왼쪽에서부터 오른쪽으로 검색 해 주세요

두번째 문자는 오른쪽에서부터 왼쪽으로 검색 해 주세요

검색해서 가장 먼저 발견되는 글자들이 얼마나 떨어져 있는지 출력하세요

만약 P와 A를 입력받았다면 정답은 3 입니다

ATKPTCABC
3칸 떨어짐

입력 예제

P A

출력 결과

3



문제 6번

win			people			
3	5	1	1	5	7	0
4	2	6				

하드코딩

입력

win 배열에 합격자 명단이 있습니다.

people의 번호를 입력 받고,

네명의 합격 여부를 출력 하주세요.

입력 예제

1 5 7 0

출력 결과

1번 합격

5번 합격

7번 불합격

0번 불합격



문제 7번

"CODING" 문장을 **vect**라는 배열에 하드코딩 해 주세요

이제 찾을 문자 개수 n 과 문자들을 입력 받습니다.

입력받은 각 문자가 **vect**배열에 존재하는지 출력 해 주세요.

존재하면 O, 존재하지 않으면 X를 출력 해 주세요.

- DAT(Direct Address Table)을 이용하시면 됩니다

ex)

5

A B C D E

출력결과 : XXOOX

입력 예제

3

I N G

출력 결과

OOO



문제 8번

세 문장을 입력받고

같은글자가 한글자라도 없으면 Perfect

아니면 No를 출력 해 주세요. (한문 장안에 중복된 글자 체크)

Ex)

ABCDEF

GHIJKLMN

OPQR

를 입력 하였으면

Perfect를 출력 해 주세요.

Ex2)

BBQWORLD

ABCWORLD

ZYM

를 입력 하였으면

No를 출력 해 주세요.

입력 예제

ABCDEF

GHIJKLMN

OPQR

출력 결과

Perfect



문제 9번

한문장을 입력 받고, 중복 알파벳을 제거한 후 **알파벳 순서대로 출력** 해주세요.

(A~F글자, 최대 10글자, **direct address table** 자료구조를 이용해주세요)
ex)

입력 : ABBACCDEA

출력 : ABCDE

입력 예제

ABBACCDEA

출력 결과

ABCDE



문제 10번

한문장을 입력받고, 각 글자마다 수를 출력 해주세요.

최대 10글자 까지 입력될 수 있습니다

(DirectAddressTable을 이용해주세요)

ex)

입력 : BTABATP

출력:

A:2

B:2

P:1

T:2

입력 예제

BTABATP

출력 결과

A:2

B:2

P:1

T:2



문제 11번

한 문장을 입력 받으세요.

한 문장에서 GHOST 단어가 존재하는지 찾아서 출력 해 주세요.

QGHOSTA를 입력 받았다면

입력받은 문장안에 GHOST 가 존재 하므로 **존재**를 출력 해 주세요.

ABGOSAT를 입력 받았다면

입력받은 문장안에 GHOST가 존재 하지 않으므로 **존재하지 않음** 을 출력 해 주세요.

입력 예제

QGHOSTA

출력 결과

존재



문제 12번

시간을 나타내는 Time 클래스를 만들어 보자. Time 클래스에는 시(int hour_), 분(int minute_), 초(int second_)를 나타내는 멤버 변수를 포함하고 있다. 다음 main 함수와 실행 결과를 참고하여 두 객체 사이의 + 연산이 가능하도록 만들어 보라.

main.cpp :

```
#include <iostream>
#include "time.h"
using namespace std;

int main() {
    Time time1(4, 50, 40);
    Time time2(3, 40, 30);
    Time time3 = time1 + time2;

    time1.Print();
    time2.Print();
    time3.Print();

    return 0;
}
```